

Llibre electrònic

Lloc: AULES
Curs: Programació
Llibre: Llibre electrònic

Imprés
per: ALEJANDRO GONZÁLVEZ TEIJEIRO
Data: divendres, 18 de setembre 2026,
16:07



Taula de continguts

1. Programari

1.1. Introducció històrica

2. El cicle de vida del programari

3. Algorismes

3.1. Diagrames de flux

3.2. Pseudocodi

4. El codi màquina

5. El llenguatge ensamblador

6. Llenguatges d'alt nivell

6.1. Llenguatges compilats

6.2. Llenguatges interpretats

6.3. Execució de "bytecodes"

7. El llenguatge Java

7.1. Estructura bàsica d'entrada i eixida



1. Programari

Els sistemes informàtics han suposat una revolució per a la manera en la que treballem a les últimes dècades, ja que ens han proporcionat l'habilitat processar i compartir informació d'una manera impensable abans de la seua aparició. Els sistemes informàtics es troben formats per la unió de dos elements complementaris: el maquinari o "hardware" amb capacitat de llegir i executar conjunts d'ordres prèviament creats expressament per a una tasca donada i, per una altra banda, tenim aquestes ordres, aplicacions o programes que reben el nom de programari o "software".

La creació del programari és la tasca que ens ocuparà al llarg d'aquest mòdul i representa una tasca eminentment lògica i que requereix d'un esforç i capacitat mental considerables, així com de rigorositat a l'hora d'organitzar les nostres idees i recursos.



1.1. Introducció històrica

Disposar d'una màquina analítica, capaç de realitzar gran quantitat de càlculs de manera automàtica, ha sigut el somni d'una gran quantitat de pensadors i científics des de fa segles: Charles Babbage, Alan Turing... Però no va ser fins al segle passat quan varem disposar de la tecnologia necessària per tal de desenvolupar les primeres computadores realment funcionals.

A l'inici de l'era informàtica els ordinadors electromecànics (com l'ENIAC, o els seus successors amb arquitectura von Neumann) ocupaven grans superfícies i consumien quantitats d'energia elevades (i produïen un soroll també realment considerable), però l'evolució dels components de procés (des dels relés, passant per les vàlvules de buit i els transistors bipolars amb lògica TTL fins als actuals circuits integrats CMOS amb velocitat de commutació ultra-alta) ha permès disposar de vertaders ordinadors per tot arreu, des de dins dels aparells de microones, passant pels mòbils fins arribar als ordinadors de sobretaula i els grans super-computadors.

Ara que la tecnologia informàtica es troba per tot arreu, el problema històric que sols preocupava a un grapat de científics ens afecta amb un major pes: com fem que les "màquines analítiques" o ordinadors realitzen la tasca que nosaltres volem? o millor dit: quin és el procés pel qual es creen les aplicacions informàtiques que ens permeten que les computadores realitzen per nosaltres les tasques que ens interessin? La resposta és el procés que anem a estudiar al llarg del mòdul: la programació.



2. El cicle de vida del programari

El cicle de vida del programari representa tots aquells passos o processos que s'han de dur a terme des de que es planteja un cert problema a resoldre mitjançant un programa informàtic fins que el mateix es troba ja instal·lat i en funcionament de cara a l'usuari. El cicle de vida comença amb l'anàlisi de les necessitats i especificacions del client o cap de projecte i es compon dels passos que tens a continuació (*figura 1*):

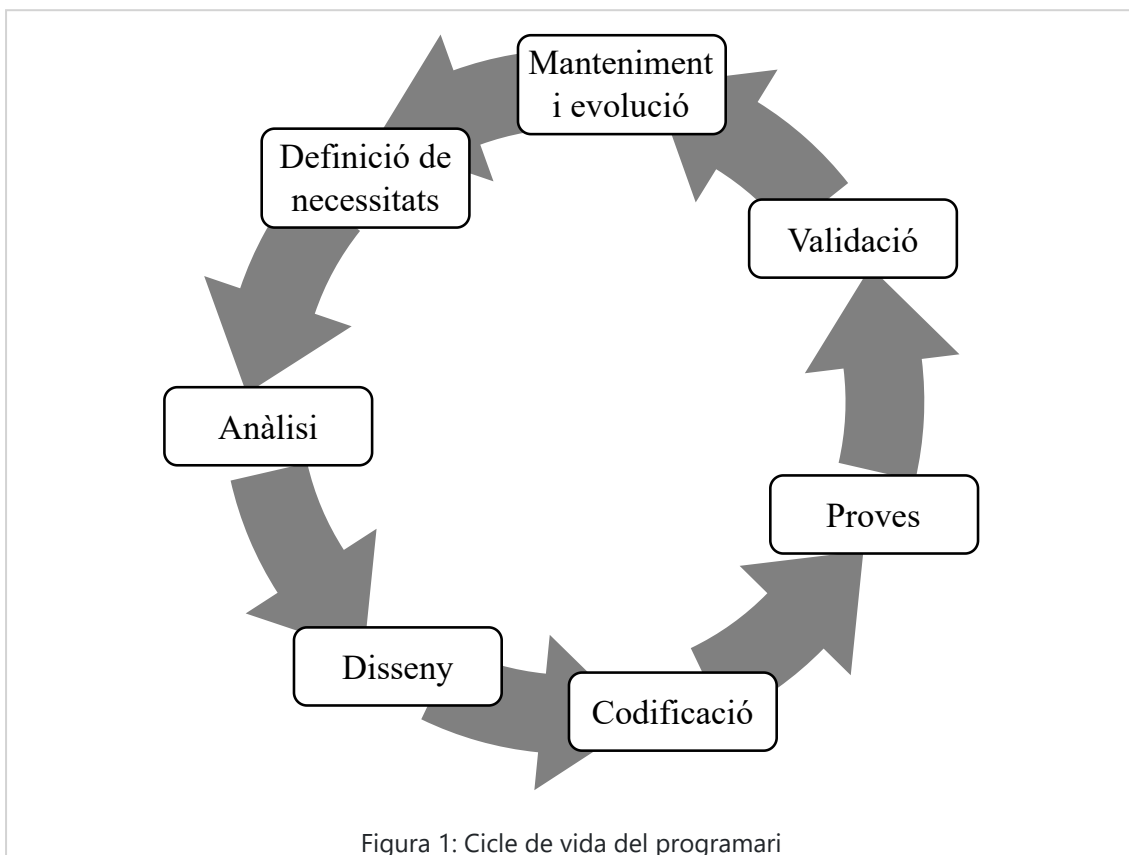


Figura 1: Cicle de vida del programari

- **Definició de necessitats:** Comencem amb una visió general d'allò que es necessita i possibles solucions al nostre problema.
- **Anàlisi:** Comencem amb una entrevista amb el client i/o definició per escrit d'allò que es necessita; cal crear una definició el més estricta i concisa possible per tal de recollir tots els requeriments de manera clara i cal procedir amb una definició exacta de les característiques del producte final, funcionament, limitacions i rendiment.
- **Disseny:** Els dissenyadors i arquitectes de programari decideixen les diferents estructures de dades, sistemes d'emmagatzemament de la informació, algorismes i interfícies d'usuari que es van a crear o a fer servir.
- **Codificació:** En aquesta etapa s'implementen les solucions fent servir el llenguatge de programació escollit i les biblioteques i eines externes triades.
- **Proves:** Realitzarem proves de diversos tipus, incloent proves unitàries, d'integració i d'acceptació per part dels usuaris.
- **Validació:** Una vegada implementada la solució cal analitzar-la per tal de decidir si s'adequadament o no als objectius previstos.
- **Manteniment i evolució:** Finalment analitzem nous requeriments que han sorgit arran de l'ús de la aplicació i tornem a començar un altre cicle de desenvolupament si es creu convenient per tal

d'obtenir una nova versió del programari que s'adapte millor als nous requeriments.

Resulta molt important començar el cicle de desenvolupament del programari de manera correcta, recollint de manera exacta els requeriments del mateix i realitzar un anàlisi de la manera més exacta possible, ja que si comencem amb el disseny i codificació a partir necessitats no ben especificades o d'un anàlisi no ben realitzat ens trobarem amb una aplicació que no s'adaptarà a allò que es necessita per part del client.



3. Algorismes

Un algorisme és un conjunt d'instruccions finit o passos concrets i no ambigus que quan es duen a terme serveixen per a executar una tasca o resoldre un problema. A la nostra vida fem servir algorismes de manera contínua i a vegades sense adonar-nos-en com per exemple: elaborar un plat culinari, multiplicar dos números, buscar el camí per anar des d'on ens trobem a allà on volem anar... El nom d'algorisme prové del matemàtic i astrònom àrab al-Khwarizmi autor d'una obra on parla de les tècniques usades per tal d'operar amb el sistema numèric aràbic (o indi) escrit l'any 825.

El nostre interès pels algorismes i l'algorísmia (estudi dels algorismes) deriva de la seua utilitat alhora de crear solucions que permeten que un sistema informàtic resolga de manera eficaç els problemes plantejats per part d'un client o que es troben a una especificació d'un programa informàtic donat. Saber crear, entendre i interpretar un algorisme per a resoldre cert problema és essencial per a qualsevol analista/programador, ja que un algorisme ben escrit pot ser traduït a qualsevol llenguatge de programació i acabar sent part d'una solució informàtica.

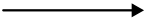
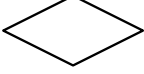
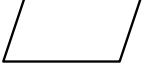
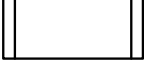
Anem a estudiar dues maneres diferents d'especificar algorismes: el pseudocodi i els diagrames de flux.



3.1. Diagrames de flux

Un diagrama de control de flux representa d'una manera gràfica un algorisme donat fent servir una sèrie de símbols amb significat ben definit per tal d'expressar les instruccions del mateix; els diferents elements gràfics es trobaran units entre ells mitjançant fletxes que connectaran els punts d'entrada i eixida de l'algorisme amb la resta de símbols, formant un camí o sèrie de camins que es van a poder recórrer d'una o diverses formes i que sempre portaran des del punt d'inici al punt de final.

Els diferents elements que trobem als diagrames de control de flux són els de la *figura 3*:

Forma	Nom	Definició
	Fletxa	Tenen la utilitat d'enllaçar els diferents elements del diagrama per tal d'indicar l'ordre de les operacions. Les puntes de fletxa són opcionals i davant la seua falta assumim que la direcció del moviment és d'esquerra a dreta i de dalt cap avall.
	Inici / final	Es representa com una el·lipse o com un rectangle amb les puntes arrodonides i ha de dur al seu interior bé la paraula "Inici" o "Final", encara que també pot aparèixer alguna descripció del procediment que comença o acaba.
	Procés	Un procés pot ser una operació o conjunt d'operacions del nostre diagrama com: una assignació o canvi de valor d'alguna variable, alguna operació aritmètica o lògica o qualsevol altra.
	Decisió	Indica una operació condicional on el flux d'execució del nostre algorisme pot prendre dos camins diferents en funció de la condició indicada.
	Entrada / eixida	S'indica amb un romboide i representa una operació en la que bé es mostra alguna informació a l'usuari, bé es demana la introducció de dades al mateix per algun mitjà com: teclat, ratolí, tauleta o altres.
	Procediment emmagatzemat	Es fa servir per tal d'indicar la invocació d'un procediment amb un nom definit descrit en algun altre lloc del mateix full o d'un altre full del projecte.
	Connector intern	Permet l'enllaç del flux d'execució amb un altre punt del mateix full en el que ens trobem.

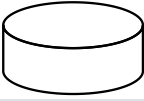
	Connector extern	Permet l'enllaç del flux d'execució amb un altre punt que es troba en un altre full del projecte.
	Fitxer	Representa la lectura o escriptura d'informació en un fitxer que es troba en un sistema d'emmagatzemament permanent.
	Base de dades	Representa la lectura o escriptura d'informació en un sistema gestor de bases de dades.

Figura 3: Elements dels diagrames de control de flux

Tot seguit tens l'algorisme per a trobar el número màxim d'una llista expressat en diagrama de control de flux i llegint la informació a través d'algun sistema d'entrada/eixida (*figura 4*):

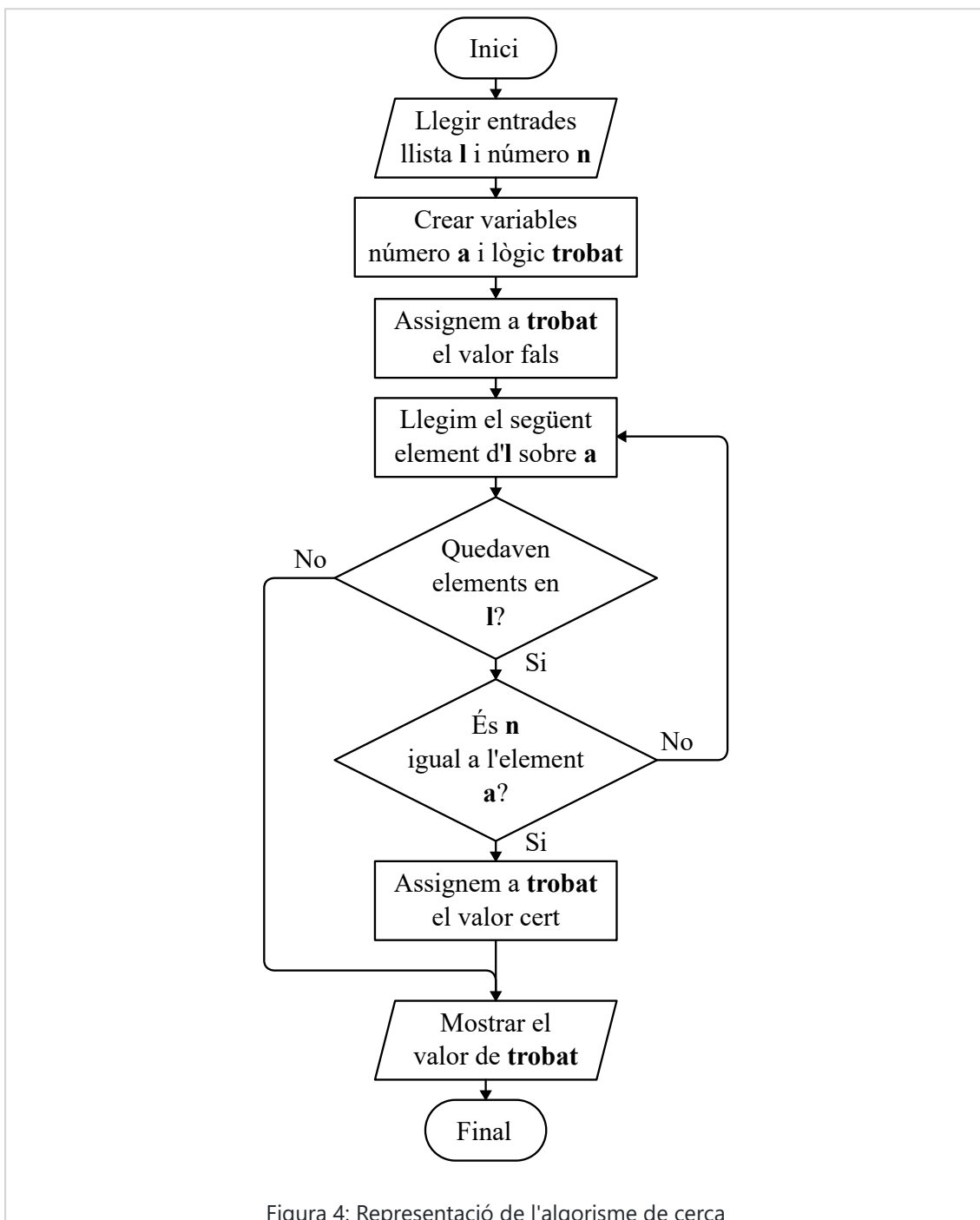


Figura 4: Representació de l'algorisme de cerca

3.2. Pseudocodi

El pseudocodi (o fals codi) és una representació de molt alt nivell de l'estructura d'un programa que fa servir estructures típiques dels llenguatges de programació però es troba pensat per a que qualsevol persona pugui entendre el codi que conté. El pseudocodi no compta amb especificacions estrictes sobre la seua notació, així que pot contindre indicacions en llenguatge natural, fórmules matemàtiques, declaracions de variables, paràmetres o dades d'entrada, definicions dels àmbits dels mateixos...

Per norma general, quan escrivim un algorisme en pseudocodi, diferenciem d'una determinada manera diferents seccions amb els paràmetres d'entrada amb els seus rangs de valors permesos, les variables que es van a fer servir i el bloc amb les instruccions o passos a seguir. Tot seguit tens un exemple de representació en pseudocodi d'un algorisme que troba si un número es troba a dins d'una llista de valors (*figura 2*):

Entrades:

llista de números l
número n

Variables:

número a
lògic trobat

Instruccions:

trobat ← fals
per a cada número a en l fer
 si (a = n) llavors trobat ← cert

Eixides:

lògic trobat

Figura 2: Algorisme d'exemple en pseudocodi



4. El codi màquina

Els ordinadors actuals no deixen de ser màquines extraordinàriament complexes de construir i també d'usar de manera directa, ja que els programes que executen es troben formats per una sèrie (bastant llarga) de "1" i "0" que formen les anomenades instruccions màquina. Una instrucció màquina és l'element bàsic d'un programa, i instrueix la CPU per tal que realitzi una tasca simple, com pot ser sumar els números emmagatzemats en un registre i emmagatzemar-ne el resultat en un altre.

L'estructura i codificació de les instruccions màquina són exclusives de cada arquitectura de CPU, és a dir, les instruccions que usen els processadors IA64 d'IntelTM no són ni paregudes a les que fa servir ARM[®]v7 per exemple; a més a més, cada arquitectura de CPU disposa de variants que augmenten o modifiquen l'anomenat joc d'instruccions disponibles (el conjunt d'instruccions que entén la CPU).

Com podràs comprovar, desenvolupar una aplicació usant el llenguatge màquina és una tasca costosa, molt lenta de realitzar, que ens obliga a conèixer exhaustivament l'estructura interna de la CPU que anem a usar i que, a més a més, ens obliga a reescriure completament cada aplicació per a totes les arquitectures sobre les que volem que s'execute.

Si en tens curiositat, ací tens una instrucció RISC-V RV32C que suma el registre 2 al registre 3 de la CPU i desa el resultat sobre el primer: "1001000100001110".



5. El llenguatge ensamblador

El llenguatge ensamblador representa una gran reducció en la dificultat d'elaborar programes dirigits a una arquitectura en concret, ja que assigna noms fàcils de recordar a cada instrucció màquina disponible per a una CPU donada. Com que un programa elaborat en ensamblador no es troba en codi màquina, aquest codi cal passar-lo a una aplicació que l'assembla, és a dir, que tradueix les instruccions ensamblador en codi binari que la CPU entén de manera directa.

Elaborar un programa en llenguatge ensamblador és molt més fàcil i ràpid que codificar instruccions màquina, però encara ens obliga a conèixer exhaustivament la CPU per a la que estem treballant y a recodificar cada aplicació per a cada arquitectura sobre la volem que s'execute.

Com a curiositat, la instrucció anterior de suma que figura al punt anterior s'expressa en llenguatge ensamblador com a: `"c.add a2, a3"`; com pots comprovar, resulta molt més fàcil d'entendre i recordar que el codi binari equivalent.



6. Llenguatges d'alt nivell

Els llenguatges d'alt nivell ens proporcionen un grau d'abstracció molt més elevat del que mai ens podrà donar l'assemblador, i ens permeten l'elaboració de programes informàtics sense preocupar-nos ni de la CPU per a la que estem treballant ni sobre les particularitats internes de la mateixa.

Els llenguatges d'alt nivell es troben formats per instruccions escrites en llenguatge natural (el que les persones parlem) i per regles que es poden usar per tal de formar expressions amb les que s'indica allò que volem que la màquina realitzi.

Una vegada tenim un programa escrit en un llenguatge d'alt nivell, ens trobem amb un problema: com fem que una CPU que sols entén codi màquina execute aquest programa? La resposta és que disposem de tres alternatives per tal de fer-ho: la compilació, la interpretació i l'execució de "bytecodes".



6.1. Llenguatges compilats

El procés de compilació és semblant al d'assemblatge, encara que resulta bastant més complex per no haver cap tipus de correspondència directa entre les instruccions que s'escriuen i el codi màquina. El procés consisteix en disposar d'una aplicació que interpreta un fitxer o fitxers amb codi d'alt nivell i fruit de l'interpretació, trau com a eixida un programa executable en codi màquina (normalment es realitza també un procés anomenat enllaçat que afegeix codi extern capaç de realitzar tasques diverses). El procés es pot observar de manera gràfica a la *figura 5*.

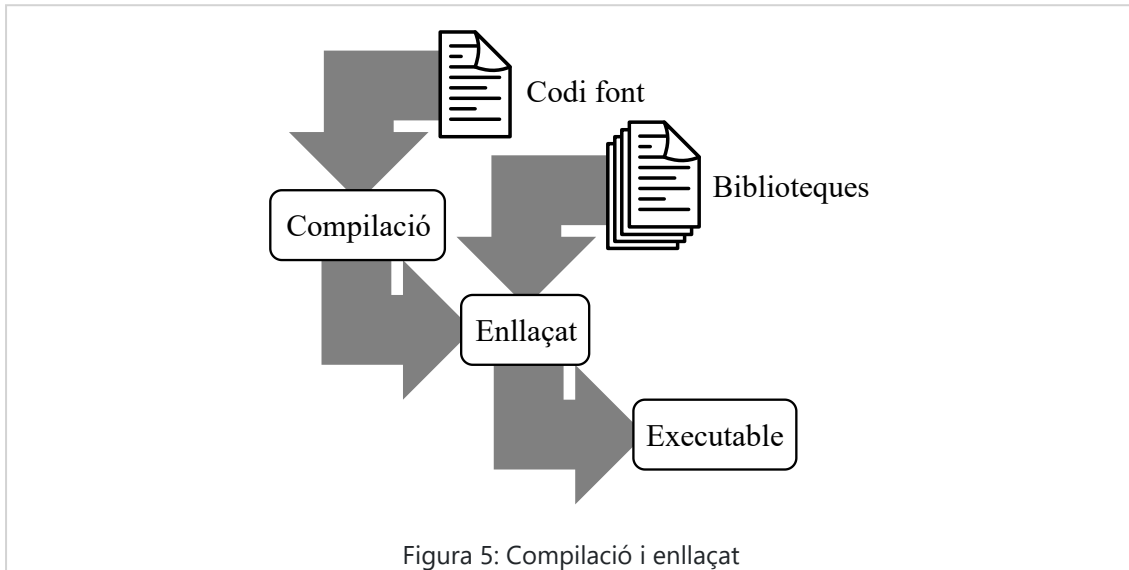


Figura 5: Compilació i enllaçat

Els principals avantatges dels llenguatges compilats es poden resumir en els següents:

- Produeixen programes eficients, ja que es troben en codi màquina directament interpretable.
- Si es disposa de compiladors per a diferents plataformes, una única aplicació es podrà executar sobre totes (típicament).
- El codi generat és difícilment llegible per part de tercers.
- El procés de compilació sols s'ha de dur a terme una única vegada, i després es podrà executar l'aplicació sempre que es vulga.

Per contra, la compilació porta implícits certs desavantatges:

- El codi generat és difícilment llegible per part de tercers (si, ho sé, també apareix a l'apartat anterior).
- Les aplicacions s'han de compilar per a cada plataforma a la que vagen dirigides.
- Trobar errors a les aplicacions pot resultar complicat.



6.2. Llenguatges interpretats

La interpretació representa una manera de treballar completament oposada a la compilació, ja que el que es fa és disposar d'un programa específic (l'interpret) que llig els fitxers que contenen el llenguatge d'alt nivell i a la vegada que intenta interpretar el codi que troba, l'executa. Al procés d'interpretació no es produeix mai cap pas de codi d'alt nivell a màquina, per tant l'interpret és un element necessari per a l'execució dels programes.

La interpretació proporciona els següents beneficis:

- Els programes solen ocupar un espai bastant reduït.
- Trobar errors al codi escrit és bastant fàcil degut a les eines d'informe de fallades de que disposen els interprets.
- Una única aplicació es pot distribuir i executar sobre totes les plataformes per a les que estiga disponible l'interpret i sense haver de preocupar-nos sobre temes d'arquitectures o altres.
- El codi d'alt nivell es troba sempre disponible per a qualsevol.

Encara que disposa també dels següents inconvenients:

- Els programes interpretats són generalment més lents que els compilats, ja que la traducció es realitza cada vegada que s'executa una aplicació.
- Per tal d'executar les aplicacions, s'ha de disposar d'un interpret ja instal·lat.
- El codi d'alt nivell es troba sempre disponible per a qualsevol.



6.3. Execució de "bytecodes"

L'execució de "bytecodes" o els llenguatges compilats-interpretats representen una tècnica a meitat camí entre la traducció i la interpretació, i amb aquesta s'intenten solucionar els problemes que tenen els dos mètodes anteriors.

La manera d'operar d'aquests llenguatges es basa en un pas de compilació, que produeix un codi binari universal (el "bytecode") que no és executable directament per cap CPU actual (encara que se'n podrien construir de compatibles), seguit d'un intèrpret (anomenat màquina virtual) que interpreta aquest codi i l'executa al mateix temps.

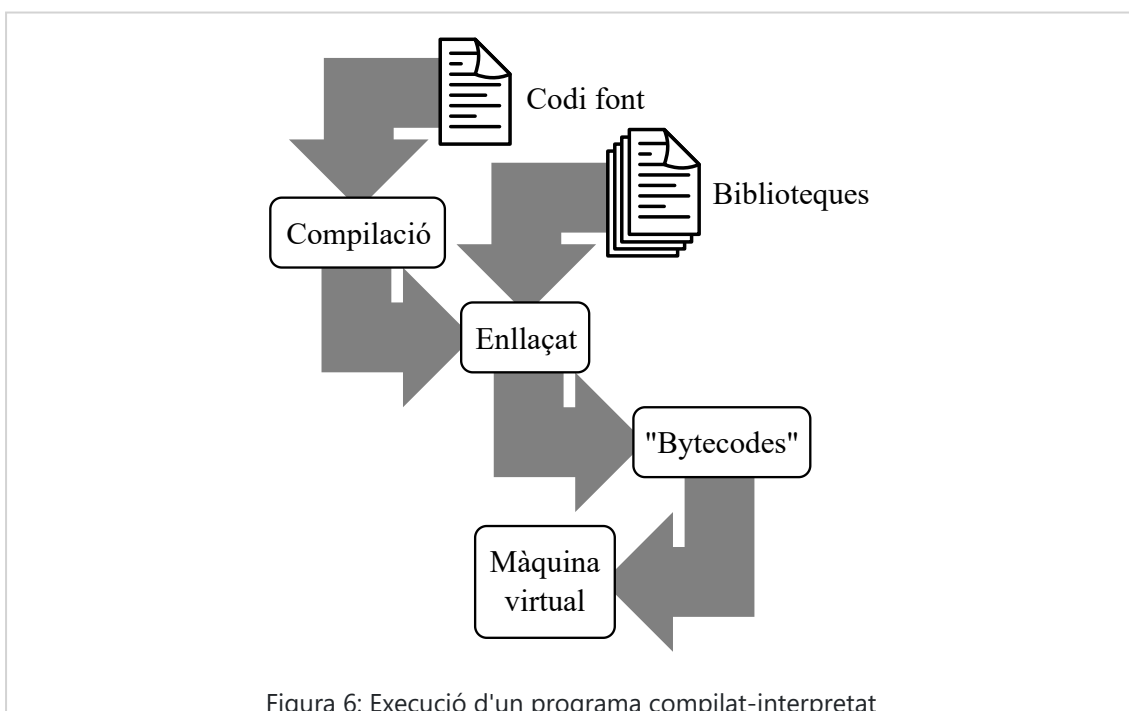
Tal com passa amb la compilació o interpretació pures, aquesta tècnica també presenta avantatges i inconvenients. Els principals avantatges són:

- Independència total de l'arquitectura a la que es troben dirigits els programes, és a dir, que si disposem d'una màquina virtual a la nostre equip per a un tipus de "bytecode" donat, llavors podrem executar-lo.
- Execució molt més ràpida que els programes interpretats.
- El codi generat és difícilment llegible per part de tercers.

Els principals inconvenients són:

- El codi generat és difícilment llegible per part de tercers.
- Execució un poc més lenta que en el cas dels programes compilats a codi natiu.
- Necessitat de disposar d'una màquina virtual ja instal·lada.

Tot seguit (*figura 6*) tens un esquema de la manera d'operar d'aquesta tècnica:



7. El llenguatge Java

Java™ és un llenguatge d'alt nivell, propòsit general i orientat a objectes creat per James Gosling el 1995. Java es troba altament influenciat pel llenguatge C, del que pren gran part de la seua sintaxi, encara que n'elimina algunes de les seues característiques més complexes, com la manipulació directa de memòria i els punters.

La manera més freqüent d'operar per a les aplicacions Java consisteix en la generació d'un codi intermedi o "bytecode" que a la seua vegada és interpretat per l'anomenada "màquina virtual de Java". La compilació directa a codi natiu també és possible fent ús de l'anomenat compilador JIT (Just In Time).

Java™ destaca per la seua versatilitat i potència, ja que permet: aplicacions de sobretaula, aplicacions en entorns servidor, applets de navegador, aplicacions mòbils i per a dispositius embastats i molts altres entorns.

A l'any 2007 la major part de les tecnologies Java™ es varen alliberar baix la llicència GNU GPL, pel que actualment es pot considerar que Java es tracta de programari lliure.

Els elements bàsics necessaris per tal de desenvolupar un programa Java™ són:

- Un editor de textos simple.
- El compilador de Java™ anomenat "**javac**".
- La màquina virtual "**java**".

El procés més bàsic d'elaboració d'una aplicació en Java™ comença amb l'escriptura d'un fitxer de text amb el font de l'aplicació amb l'extensió ".java". Tot seguit s'invoca el compilador que rep el fitxer font i genera el "bytecode" del mateix amb l'extensió ".class". Finalment la màquina virtual "**java**" executa el "bytecode".



7.1. Estructura bàsica d'entrada i eixida

En aquesta primera unitat anem a començar a escriure programes en Java™, encara que sense preocupar-nos massa d'aspectes complexos que ja anirem veient a les pròximes. Per ara sempre que vulguem realitzar una aplicació començarem editant un fitxer de text pla que rebrà el nom de l'aplicació que vulguem crear, com ara "hola.java" i tot seguit escriurem el que apareix a la figura següent (figura 7):

```
public class hola {  
    public static void main (String[] args) {  
        System.out.println ("HoLa món");  
    }  
}
```

Figura 7: hola.java, la nostra primera aplicació

Una vegada acabat el fitxer, el compilarem amb el comandament "javac hola.java" i l'executarem amb "java hola". Ja hem compilat i executat el nostre primer programa!

Si el que volem és llegir informació per teclat, llavors hem d'usar variables (si vols saber què són, avança a la següent unitat) per tal de desar les dades. Ací tens (figura 8) el codi del nostre segon programa "salutacio.java":

```
import java.io.*;  
public class salutacio {  
    public static void main (String[] args) {  
        BufferedReader br = new BufferedReader (new InputStreamReader  
(System.in));  
        String nom = "";  
        System.out.println ("Com et dius?");  
        try {  
            nom = br.readLine();  
        }  
        catch (Exception ex) {}  
        System.out.println ("HoLa" + " " + nom);  
    }  
}
```

Figura 8: salutacio.java, la nostra segona aplicació

El programa el compilarem i executarem d'igual manera que hem fet amb el primer.



És important notar que l'ordre "**System.out.println**" imprimeix per pantalla tot el text que se li passe entre cometes i el contingut de les variables que se li indiquen.